

FRENCH UNIVERSITY IN ARMENIA

Faculty of Computer science and Applied mathematics

Subject Database | Database Modeling and Implementation

Title Hospital Pandemic Management System

Student Lia SARGSYAN, Madlen MINASYAN, Milena YEGANYAN

Teacher Varazdat AVETISYAN, PhD

April 4, 2026

Table of Contents

1. Problem Description and Requirements Analysis	4
1.1 System Overview	4
1.2 Problem Definition.....	5
1.3 Objectives of the System	5
1.4 Functional and non-functional Requirements.....	6
1.6 Stakeholders / Users.....	7
2. Conceptual Design	8
2.1 Entities	8
2.2 Relationships.....	9
3. Logical Design	11
3.1 Relational Schema	11
3.2 Keys	12
3.3 Constraints.....	14
4. Normalization	15
4.1 First Normal Form (1NF).....	15
4.2 Second Normal Form (2NF)	15
4.3 Third Normal Form (3NF)	15
5. Relational Algebra	16

5.1 Selection (σ).....	16
5.2 Projection (π)	16
5.3 Join ($\bowtie$).....	17
5.4 Union ($\cup$)	17
5.5 Intersection ($\cap$)	17
5.6 Difference ($-$)	18
6. Physical Implementation	19
6.1 Data Definition Language (DDL).....	19
6.2 Data Manipulation Language (DML)	19
6.3 Data Query Language (DQL)	20
6.4 Views	20
6.5 Indexes	20
6.6 Trigger.....	21
6.7 Stored Procedure.....	21
6.8 Access Control.....	22
7. Results and Testing.....	23
8. Conclusion	24

1. Problem Description and Requirements Analysis

1.1 System Overview

The Hospital Pandemic Management System is a relational database designed to manage and organize patient-related data during infectious disease outbreaks. Its main purpose is to provide a centralized and structured environment for storing important medical and administrative information. The system supports pandemic-related monitoring by handling data such as patient records, laboratory test results, vaccination history, infection status, and contact tracing information.

The system addresses the need for efficient data management in healthcare environments, especially during periods of increased epidemiological risk. It enables healthcare professionals to quickly access and analyze data, identify infected or high-risk patients, and track possible transmission between individuals. By integrating multiple aspects of hospital operations into a single database, the system reduces data redundancy and improves consistency.

From a functional perspective, the system allows storing and updating patient information, recording medical tests and vaccinations, tracking infection status, managing hospital visits, and analyzing patient interactions. As a result, it supports both daily hospital operations and informed decision-making during critical situations.

1.2 Problem Definition

Healthcare institutions often face significant challenges in managing and tracking patient data, especially during infectious disease outbreaks. In many cases, information is distributed across multiple systems or recorded in an unstructured manner, which makes it difficult to access, update, and analyze efficiently. This lack of integration leads to delays in decision-making and increases the risk of errors or incomplete data.

A major problem is the inability to effectively track the spread of infections within a population. Without a centralized system, hospitals struggle to monitor patient status, identify high-risk individuals, and trace contacts between patients who may have been exposed to a virus. Additionally, the absence of structured data management makes it difficult to maintain accurate records of vaccinations, laboratory test results, and hospital visits.

Therefore, there is a clear need for a unified database system that centralizes all relevant medical information, ensures data consistency, and enables efficient tracking and analysis of infection-related data.

1.3 Objectives of the System

The main objectives of the system are:

- To manage and store patient personal and medical data in a structured manner
- To track and maintain vaccination records for each patient
- To monitor and store laboratory test results

- To track and update the infection status of patients
- To identify and manage patient risk factors (e.g., chronic conditions)
- To enable efficient contact tracing between patients
- To record and manage hospital visits and treatments
- To ensure accurate, consistent, and centralized data management
- To support healthcare professionals in data analysis and decision-making

1.4 Functional and non-functional Requirements

The functional requirements define the main operations that the system must perform.

The system must:

- Store and manage patient personal information
- Record and update vaccination data
- Record and maintain laboratory test results
- Track and update the virus infection status of patients
- Store and manage patient risk factors
- Track and record contact interactions between patients
- Record and manage hospital visits and related details
- Allow efficient querying, retrieval, and analysis of data

The non-functional requirements define the quality attributes of the system. The system must ensure:

- **Data accuracy:** all stored information must be correct and consistent
- **Security:** access to sensitive data must be controlled and protected
- **Reliability:** the system should function correctly without failures
- **Performance:** queries and operations should be executed efficiently
- **Scalability:** the system should handle increasing amounts of data over time

1.6 Stakeholders / Users

The system is designed for the following main users:

- **Doctor (Administrator):** has full access to the database, including the ability to insert, update, and retrieve patient data, manage laboratory test results, vaccination records, infection status, and hospital visits
- **Medical Staff (User):** has limited access, mainly focused on viewing patient information and assisting in updating certain records under supervision

2. Conceptual Design

2.1 Entities

The system is composed of the following main entities:

- **Patient** → stores personal and identification information of each patient
- **Residence** → contains address and living condition details of patients
- **Staff** → represents hospital employees such as doctors and medical personnel
- **Vaccination** → stores vaccination history and related details for each patient
- **Lab_Test** → contains laboratory test results, including test type and outcomes
- **VirusStatus** → tracks the infection status and quarantine condition of patients
- **RiskFactor** → records medical conditions and health risks associated with patients
- **ContactRecord** → stores interactions between patients for contact tracing purposes
- **HospitalVisit** → records patient visits, including admission details and reasons

2.2 Relationships

- **Patient** → **Residence (many-to-one)**: each patient is associated with one residence, while one residence can be linked to multiple patients
- **Patient** → **Vaccination (one-to-many)**: a patient can have multiple vaccination records, but each record belongs to one patient
- **Patient** → **Lab_Test (one-to-many)**: a patient can undergo multiple laboratory tests, each linked to a single patient
- **Patient** → **VirusStatus (one-to-many)**: a patient can have multiple records representing changes in infection status over time

- **Patient** → **Residence (many-to-one)**: each patient is associated with one residence, while one residence can be linked to multiple patients
- **Patient** → **Vaccination (one-to-many)**: a patient can have multiple vaccination records, but each record belongs to one patient
- **Patient** → **Lab_Test (one-to-many)**: a patient can undergo multiple laboratory tests, each linked to a single patient
- **Patient** → **VirusStatus (one-to-many)**: a patient can have multiple records representing changes in infection status over time

- **Patient → RiskFactor (one-to-many):** a patient may have multiple risk factors, such as different medical conditions
- **Patient → HospitalVisit (one-to-many):** a patient can have multiple hospital visits recorded in the system
- **Patient ↔ Patient (many-to-many via ContactRecord):** patients can have contact with multiple other patients, and each interaction is recorded in the ContactRecord entity
- **Staff → Lab_Test (one-to-many):** a staff member can perform multiple laboratory tests
- **Staff → HospitalVisit (one-to-many):** a staff member can be responsible for multiple hospital visits

3. Logical Design

3.1 Relational Schema

The relational schema of the Hospital Pandemic Management System is composed of the following tables:

- **Residence**(residence_id, country, city, district, address_line, living_condition)
- **Patient**(patient_id, national_id, first_name, last_name, date_of_birth, gender, residence_id)
- **Staff**(staff_id, full_name, role, department)
- **Vaccination**(vaccination_id, patient_id, vaccine_name, dose_number, vaccination_date, batch_number)
- **Lab_Test**(test_id, patient_id, staff_id, test_type, virus_name, test_date, result)
- **VirusStatus**(status_id, patient_id, virus_name, current_status, quarantine_status, status_date)
- **RiskFactor**(risk_id, patient_id, condition_name, severity, notes)
- **ContactRecord**(contact_id, patient_id, contacted_patient_id, contact_date, contact_type, duration_minutes, risk_level)
- **HospitalVisit**(visit_id, patient_id, staff_id, admission_date, discharge_date, department, reason_for_visit)

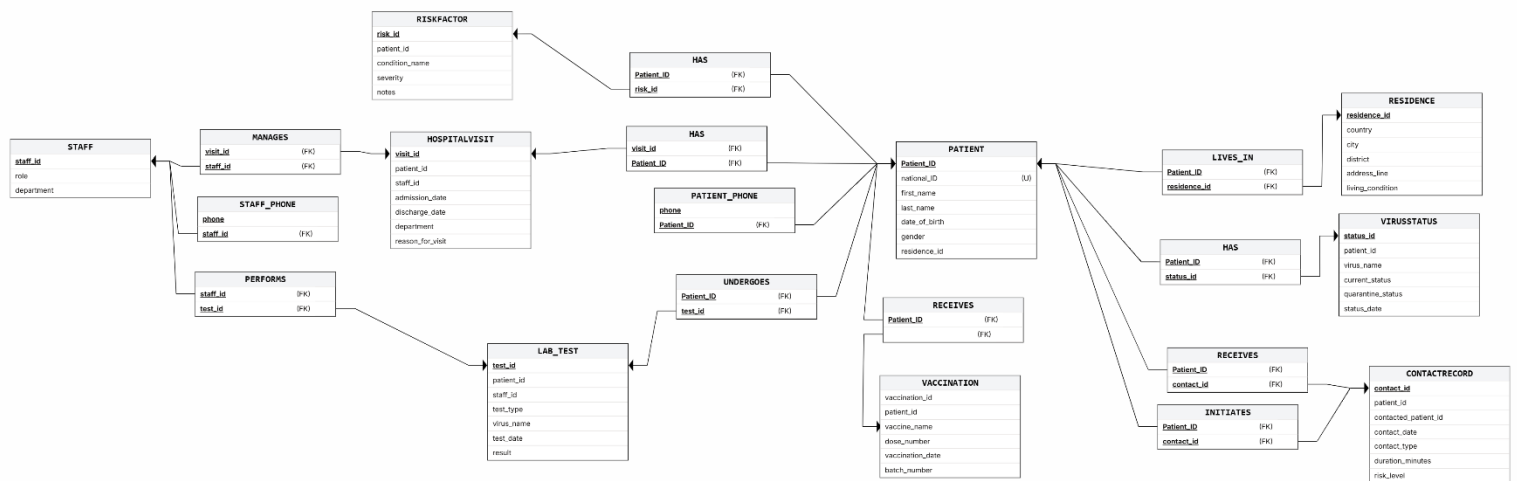


Figure 2:Relaton Schema

3.2 Keys

Primary Keys

Each table contains a primary key used to uniquely identify each record:

- Residence → residence_id
- Patient → patient_id
- Staff → staff_id
- Vaccination → vaccination_id
- Lab_Test → test_id
- VirusStatus → status_id
- RiskFactor → risk_id

- ContactRecord → contact_id
- HospitalVisit → visit_id

Foreign Keys

Foreign keys are used to establish relationships between tables:

- Patient.residence_id → Residence.residence_id
- Vaccination.patient_id → Patient.patient_id
- Lab_Test.patient_id → Patient.patient_id
- Lab_Test.staff_id → Staff.staff_id
- VirusStatus.patient_id → Patient.patient_id
- RiskFactor.patient_id → Patient.patient_id
- ContactRecord.patient_id → Patient.patient_id
- ContactRecord.contacted_patient_id → Patient.patient_id
- HospitalVisit.patient_id → Patient.patient_id
- HospitalVisit.staff_id → Staff.staff_id

Figure 3: Example of Primary and Foreign Keys in SQL Implementation

```

26
27 CREATE TABLE Patient (
28     patient_id INT IDENTITY(1,1) PRIMARY KEY,
29     national_id VARCHAR(50) NOT NULL UNIQUE,
30     first_name VARCHAR(100) NOT NULL,
31     last_name VARCHAR(100) NOT NULL,
32     date_of_birth DATE NOT NULL,
33     gender VARCHAR(10) NOT NULL,
34     residence_id INT,
35     CONSTRAINT FK_Patient_Residence
36         FOREIGN KEY (residence_id) REFERENCES Residence(residence_id)
37 );
38 GO

```

Figure 3: Example of Primary and Foreign Keys in SQL Implementation

3.3 Constraints

To ensure data integrity and consistency, the following constraints were applied in the database design:

- **PRIMARY KEY** constraints were used to uniquely identify each record in every table
- **FOREIGN KEY** constraints were used to maintain referential integrity between related tables
- **NOT NULL** constraints were applied to essential attributes to prevent missing values
- **UNIQUE** constraints were used where necessary, such as for patient national identification
- These constraints help maintain **data consistency**, prevent invalid relationships, and ensure that the database remains reliable and accurate

4. Normalization

4.1 First Normal Form (1NF)

The database satisfies the First Normal Form as all attributes contain atomic (indivisible) values, and there are no repeating groups or multivalued attributes within any table.

4.2 Second Normal Form (2NF)

The database satisfies the Second Normal Form since all non-key attributes are fully functionally dependent on the primary key. Each table represents a single entity, and there are no partial dependencies.

4.3 Third Normal Form (3NF)

The database satisfies the Third Normal Form as there are no transitive dependencies. All non-key attributes depend only on the primary key, ensuring minimal redundancy and improved data integrity.

5. Relational Algebra

Relational algebra operations are used to formally describe queries performed on the database. The following operations demonstrate how data can be retrieved and manipulated within the system.

5.1 Selection (σ)

Selects rows that satisfy a given condition.

Example:

σ gender = 'Female' (Patient)

SQL:

```
496 SELECT * FROM Patient WHERE gender = 'Female';
497 GO
```

Figure 4:

5.2 Projection (π)

Selects specific columns from a table.

Example:

π first_name, last_name (Patient)

```
492
493 SELECT first_name, last_name FROM Patient;
494 GO
495
```

Figure 5:

5.3 Join ($\bowtie$)

Combines related data from multiple tables.

Example:

Patient $\bowtie$ RiskFactor

SQL:

```
519
520     SELECT p.first_name, p.last_name, r.condition_name
521     FROM Patient p
522     JOIN RiskFactor r ON p.patient_id = r.patient_id;
523     GO
```

Figure 6:

5.4 Union ($\cup$)

Combines results from two queries without duplicates.

Example:

Patients with high or medium risk

5.5 Intersection ($\cap$)

Returns common results between two queries.

Example:

Patients with risk factors AND virus status

5.6 Difference (-)

Returns results from one query that are not in another.

Example:

Patients without vaccination

6. Physical Implementation

6.1 Data Definition Language (DDL)

The database structure was created using DDL statements. Tables were defined using CREATE TABLE statements, including appropriate data types and constraints such as **PRIMARY KEY**, **FOREIGN KEY**, and **NOT NULL** to ensure data integrity and consistency.

```

27 CREATE TABLE Patient (
28     patient_id INT IDENTITY(1,1) PRIMARY KEY,
29     national_id VARCHAR(50) NOT NULL UNIQUE,
30     first_name VARCHAR(100) NOT NULL,
31     last_name VARCHAR(100) NOT NULL,
32     date_of_birth DATE NOT NULL,
33     gender VARCHAR(10) NOT NULL,
34     residence_id INT,
35     CONSTRAINT FK_Patient_Residence
36         FOREIGN KEY (residence_id) REFERENCES Residence(residence_id)
37 );
38 GO

```

Figure 7: Example of Table Creation with Constraints

6.2 Data Manipulation Language (DML)

Data was inserted into all tables using INSERT statements. Each table contains more than 40 records, satisfying the project requirement. Data generation was automated using SQL queries to efficiently populate the database.

```

163
164     SELECT COUNT(*) AS total_patients FROM Patient;
165     GO
166

```

Figure 8: Data Insertion Verification (More than 40 Records)

6.3 Data Query Language (DQL)

Data retrieval was performed using SELECT queries. Various operations were implemented, including filtering conditions, **JOIN** operations to combine data from multiple tables, and **GROUP BY** clauses for aggregation and analysis.

6.4 Views

A view named **HighRiskInfected** was created to display patients who are both infected and have associated risk factors. This view simplifies complex queries and allows easier access to important analytical data.

```

576
577     SELECT * FROM HighRiskInfected;
578     GO
579

```

Figure 9: View Result – HighRiskInfected

6.5 Indexes

Indexes were implemented on frequently used columns to improve query performance. They reduce the time required for data retrieval, especially when working with large datasets.

6.6 Trigger

A trigger was implemented on the **Lab_Test** table. When a new test result with a value of *Positive* is inserted, the trigger automatically adds a corresponding record into the **VirusStatus** table. This ensures consistency and automates data updates

```

145 CREATE TRIGGER trg_after_lab_test_insert
146 ON Lab_Test
147 AFTER INSERT
148 AS
149 BEGIN
150     SET NOCOUNT ON;
151
152     INSERT INTO VirusStatus (patient_id, virus_name, current_status, quarantine_status, status_date)
153     SELECT
154         i.patient_id,
155         i.virus_name,
156         'Infected',
157         'Required',
158         CAST(GETDATE() AS DATE)
159     FROM inserted i
160     WHERE i.result = 'Positive';
161 END;
162 GO

```

Figure 10: Trigger Execution Result

6.7 Stored Procedure

A stored procedure named **GetPatientHistory** was developed to retrieve all relevant information for a specific patient. It returns data from multiple related tables, including patient details, vaccinations, lab tests, risk factors, hospital visits, and contact records.

```

590 EXEC GetPatientHistory @pid = 5;
591 GO

```

Figure 11: Stored Procedure Output

6.8 Access Control

A role named **doctor_role** was created to manage database access. Appropriate permissions such as **SELECT**, **INSERT**, and **UPDATE** were granted to ensure secure and controlled interaction with the database.

7. Results and Testing

The implemented database system was successfully tested to ensure correctness and functionality. All tables were populated with more than 40 records, satisfying the project requirements. Various SQL queries were executed, including selection, join, and aggregation operations, and all returned correct and expected results.

The trigger was tested by inserting positive laboratory test results, which automatically generated corresponding records in the **VirusStatus** table, confirming its correct behavior. The stored procedure **GetPatientHistory** was also executed successfully and returned complete and accurate information for the selected patient. Overall, the system demonstrated reliable performance and correct implementation of all required features.

8. Conclusion

The Hospital Pandemic Management System was successfully designed and implemented as a relational database. The system meets all functional and non-functional requirements, providing efficient storage, management, and retrieval of healthcare data. It ensures data consistency, supports infection tracking, and enables analytical querying.

The database is scalable and can handle increasing amounts of data over time. Additionally, it can be further extended by integrating a user interface or connecting it to real-world healthcare systems, making it a practical and adaptable solution.